

Project Specification: boxpkg

Minimalist Workstation Sandbox for Untrusted Package Installs & Agentic Loops

1. Executive Summary & Problem Statement

Ecosystems execute arbitrary code at install time (postinstall, setup.py, build.rs). Standard runs provide unconfined read access to ~/.ssh, ~/.aws, macOS Keychain, and outbound network sockets. boxpkg provides instant-startup zero-VM confinement.

2. Confinement Specifications

- **Filesystem Confinement:** Only \$PWD and isolated /tmp are writable. Secrets dotfiles blocked.
- **Keychain & Secrets Isolation:** Mach IPC lookups to security/keychain daemons denied on macOS.
- **Environment Sanitization:** Automatic stripping of environment variables matching *TOKEN*, *KEY*, AWS_.*.
- **Network Egress Modes:** Offline airgap (--airgap) or restricted registry-only HTTPS egress.

3. Isolation Matrix

Target Resource	Permission	Rationale & Implementation
\$PWD (Workspace)	Read-Write	Package managers must write node_modules, lockfiles, artifacts.
/tmp/boxpkg-	Read-Write	Ephemeral scratch directory for intermediate compiler objects.
~/.ssh, ~/.aws, ~/.gnupg	BLOCKED (DENY)	Complete denial of host secrets harvesting.
macOS Keychain / Daemons	BLOCKED (DENY)	Deny Mach service lookups to security agents.
Toolchains (/usr, /bin, /lib)	Read-Only	Compilers and runtimes execute without modification rights.
Outbound Network	Restricted	Only known package registry HTTPS endpoints permitted.

4. Implementation Roadmap (8 Weeks)

- **Phase 1 (Weeks 1–3):** macOS Seatbelt profile synthesis (sandbox-exec), environment stripper.
- **Phase 2 (Weeks 4–6):** Linux engine using unprivileged namespaces + Bubblewrap / Landlock LSM.
- **Phase 3 (Weeks 7–8):** Registry-proxy egress filter, IDE integrations, and automated vetpkg handoff.